# The Banker's Algorithm

## 1 Main Idea

**Banker's Algorithm**

The **Banker's Algorithm** is a deadlock-avoidance algorithm. Before granting a resource request, it checks whether the resulting state is safe.

The algorithm was introduced by Dijkstra. It is based on the idea of a banker who gives loans only when the bank can still satisfy all customers in some safe order.

**Decision Rule**

Grant a request only if granting it keeps the system in a safe state. If the request would create an unsafe state, postpone it.

## 2 Banking Analogy

| **Banking term** | Operating-system meaning |
|---|---|
| **Customer** | Process. |
| **Credit unit** | Resource instance, such as a tape drive or printer. |
| **Maximum credit line** | Maximum number of resources the process may need. |
| **Current loan** | Resources currently allocated to the process. |
| **Banker** | Operating system. |

## 3 Single-Resource Version

Assume there is only one type of resource. Each process declares its maximum possible need before running.

**Single-Resource Need**

$$\boxed{\text{Need}_i = \text{Max}_i - \text{Has}_i}$$

A state is safe if there is some order in which each process can receive its remaining need, finish, and release its resources.

## 4 Single-Resource Example

| | Has | Max |
|---|---|---|
| **A** | 0 | 6 |
| **B** | 0 | 5 |
| **C** | 0 | 4 |
| **D** | 0 | 7 |

**Free:** 10

| | Has | Max |
|---|---|---|
| **A** | 1 | 6 |
| **B** | 1 | 5 |
| **C** | 2 | 4 |
| **D** | 4 | 7 |

**Free:** 2

| | Has | Max |
|---|---|---|
| **A** | 1 | 6 |
| **B** | 2 | 5 |
| **C** | 2 | 4 |
| **D** | 4 | 7 |

**Free:** 1

(a) initial safe state (b) safe state (c) unsafe state

**Figure:** Three single-resource allocation states. State (c) is unsafe.

**Why state (b) is safe**

In state (b), the banker has 2 free units. Process $C$ needs only 2 more units, so $C$ can finish and release all 4 units it holds. Then another process can finish, and eventually all can complete.

**Why state (c) is unsafe**

If $B$'s request for one more unit is granted, only 1 unit remains free. No process can be guaranteed to receive enough resources to finish if all processes request their maximum needs.

## 5 Single-Resource Algorithm

**Banker's Algorithm: One Resource Type**

```
when process Pi requests k resources:
    if k > Need_i:
        reject the request

    if k > Free:
        make Pi wait

    pretend to grant the request:
        Has_i = Has_i + k
        Free = Free - k

    if the resulting state is safe:
        grant the request
    else:
        undo the pretend allocation
        make Pi wait
```

## 6 Testing Safety

**Safety Test**

```
Work = Free
mark all processes unfinished

repeat:
    find an unfinished process Pi with Need_i <= Work

    if such a process exists:
        Work = Work + Has_i
        mark Pi finished
    else:
        stop
```

```
if all processes are marked finished:
    the state is safe
else:
    the state is unsafe
```

**Meaning**

The safety test asks: "Can some process finish with the currently available resources? If yes, assume it finishes and returns its resources. Repeat."

## 7 Multiple-Resource Version

For multiple resource types, use the same idea with vectors and matrices.

| **Symbol** | Meaning |
|---|---|
| $E$ | Existing resource vector: total resources of each type. |
| $P$ | Possessed resource vector: total resources currently allocated. |
| $A$ | Available resource vector: resources currently free. |
| $C$ | Current allocation matrix: resources currently assigned to each process. |
| $R$ | Remaining request matrix: resources each process still needs to finish. |

**Vector Relationship**

$$\boxed{A = E - P}$$

## 8 Multiple-Resource Example

The resource types are tape drives, plotters, printers, and Blu-ray drives.

**Resources assigned $C$**

| | Tape | Plotter | Printer | Blu-ray |
|---|---|---|---|---|
| A | 3 | 0 | 1 | 1 |
| B | 0 | 1 | 0 | 0 |
| C | 1 | 1 | 1 | 0 |
| D | 1 | 1 | 0 | 1 |
| E | 0 | 0 | 0 | 0 |

**Resources still needed $R$**

| | Tape | Plotter | Printer | Blu-ray |
|---|---|---|---|---|
| A | 1 | 1 | 0 | 0 |
| B | 0 | 1 | 1 | 2 |
| C | 3 | 1 | 0 | 0 |
| D | 0 | 0 | 1 | 0 |
| E | 2 | 1 | 1 | 0 |

$$E = (6, 3, 4, 2), \qquad P = (5, 3, 2, 2), \qquad A = (1, 0, 2, 0)$$

## 9 Multiple-Resource Safety Algorithm

**Banker's Algorithm: Multiple Resources**

```
Work = A
mark all processes unfinished

repeat:
    find an unfinished process Pi such that Ri <= Work

    if such a process exists:
        Work = Work + Ci
        mark Pi finished
    else:
        stop

if every process is marked finished:
    the state is safe
else:
    the state is unsafe
```

**Vector comparison**

$R_i \leq Work$ means every resource need in row $R_i$ is less than or equal to the corresponding available amount in $Work$.

## 10 Example Decision

**Request from $B$**

Suppose process $B$ requests one printer. The system pretends to grant the request and then checks whether the resulting state is still safe.

This request can be granted if some process can still finish first after the pretend allocation. In the example, process $D$ can finish, returning its resources. Then other processes can finish afterward.

**Request from $E$**

After giving $B$ one printer, suppose $E$ requests the last remaining printer.

Granting $E$'s request would leave:

$$\boxed{A = (1, 0, 0, 0)}$$

No process can be guaranteed to finish from that state, so $E$'s request must be postponed.

## 11 Why the Algorithm Is Rarely Used

**Practical limitation**

The Banker's Algorithm is theoretically strong but rarely used directly in real operating systems.

| Problem | Explanation |
|---|---|
| **Maximum needs are hard to know** | Processes usually cannot predict their maximum resource needs before running. |
| **Processes change dynamically** | Users log in and out, and processes start and exit continuously. |
| **Resources can disappear** | Devices may fail, so the available resource set is not perfectly fixed. |
| **Overhead and complexity** | Repeated safety checks can be costly and inconvenient. |

**Real-world idea**

Some systems use similar heuristics. For example, a network may throttle traffic when buffer use is high, keeping enough free capacity for current users to finish.